LAPORAN PRAKTIKUM

WORKSHEET MODUL 4: PENYIMPANAN DATA PADA WEB

Login Password Terenkripsi, Keranjang Belanja Tanpa Login & Program Toko Online

Mata Kuliah: Proyek 3 - Pengembangan Perangkat Lunak Berbasis Web



Disusun Oleh:

Salman alfarisi firdaus (251511030)

2A D3

Jurusan Teknik Komputer dan Informatika

Program Studi D-3 Teknik Informatika

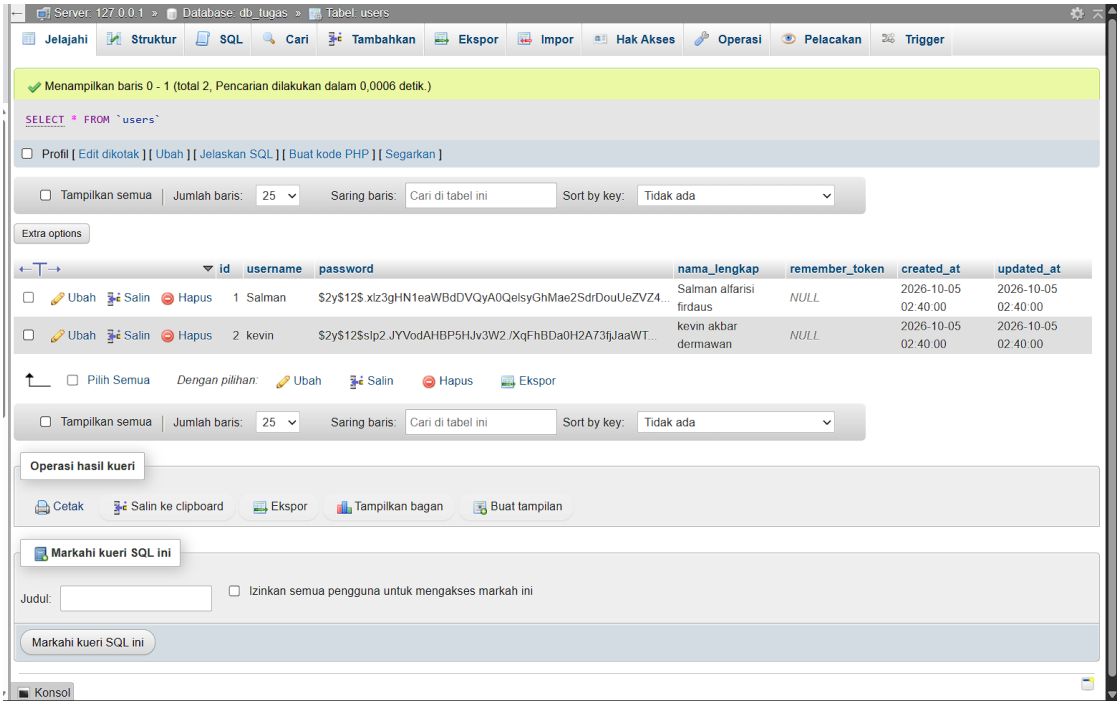
Politeknik Negeri Bandung

2026

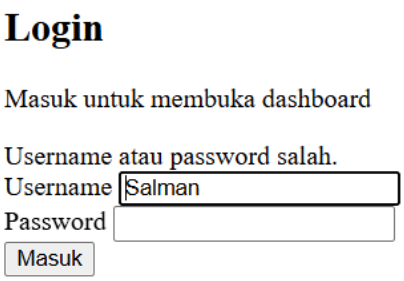
BAGIAN 1: TUGAS 1 - LOGIN DENGAN PASSWORD TERENKRIPSI

1.1 Hasil Implementasi & Tangkapan Layar (Screenshots)

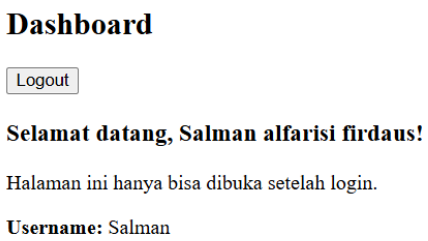
Tugas 1 mengimplementasikan sistem autentikasi mandiri pada Laravel 13 tanpa starter kit. Sistem menggunakan password hashing bcrypt, proteksi middleware auth dan guest, serta regenerasi session saat login.



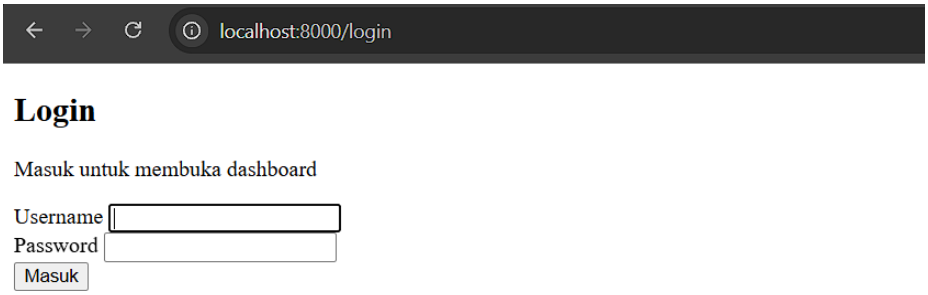
Gambar 1. Isi tabel users di phpMyAdmin (password tersimpan dalam bentuk hash $2y$12$...)



Gambar 2. Tampilan form login saat autentikasi gagal ("Username atau password salah.")



Gambar 3. Halaman dashboard setelah login berhasil menampilkan nama lengkap pengguna



Gambar 4. Halaman login setelah logout atau saat dashboard diakses tanpa login

1.2 Jawaban Pertanyaan Tugas 1

a. Mengapa password harus di-hash? Lihat kolom password di tabel users dan jelaskan isinya.

Password harus di-hash agar tidak tersimpan dalam bentuk teks biasa (plain text). Tujuannya adalah jika database bocor atau diakses pihak tidak berwenang, password asli milik pengguna tetap aman dan tidak dapat dibaca langsung. Pada kolom password tabel users (Gambar 1), isinya berupa string acak seperti $2y$12$xlz3gHN1eaWBdDVQyA... yang merupakan hasil enkripsi satu arah algoritma Bcrypt dengan cost factor 12 dan salt acak unik.

b. Mengapa logout memakai metode POST dengan @csrf, bukan link biasa?

Logout wajib memakai metode POST dengan @csrf untuk melindungi aplikasi dari serangan CSRF (Cross-Site Request Forgery). Jika logout memakai link biasa (GET), penyerang bisa menyisipkan tag gambar seperti <img src="http://localhost:8000/logout"> di halaman web lain, sehingga saat pengguna membuka halaman tersebut akun mereka akan otomatis ter-logout tanpa disengaja.

c. Apa fungsi middleware auth dan guest? Apa yang terjadi jika /dashboard dibuka tanpa login?

Middleware auth berfungsi memastikan hanya pengguna yang sudah terautentikasi (login) yang dapat mengakses route tersebut. Middleware guest berfungsi membatasi halaman (seperti form login) agar hanya bisa dibuka oleh pengguna yang belum login; jika sudah login maka akan dialihkan ke dashboard. Jika /dashboard dibuka tanpa login, middleware auth akan langsung menolak akses dan mengalihkan pengguna ke halaman /login.

d. Apa fungsi session()->regenerate() setelah login berhasil?

Fungsinya adalah mengganti Session ID lama dengan Session ID baru yang digenerate ulang oleh server setelah proses login valid. Hal ini bertujuan untuk mencegah serangan Session Fixation, yaitu serangan di

mana pihak ketiga menetapkan Session ID tertentu kepada korban sebelum login dan memanfaatkannya untuk membajak akun korban setelah login.

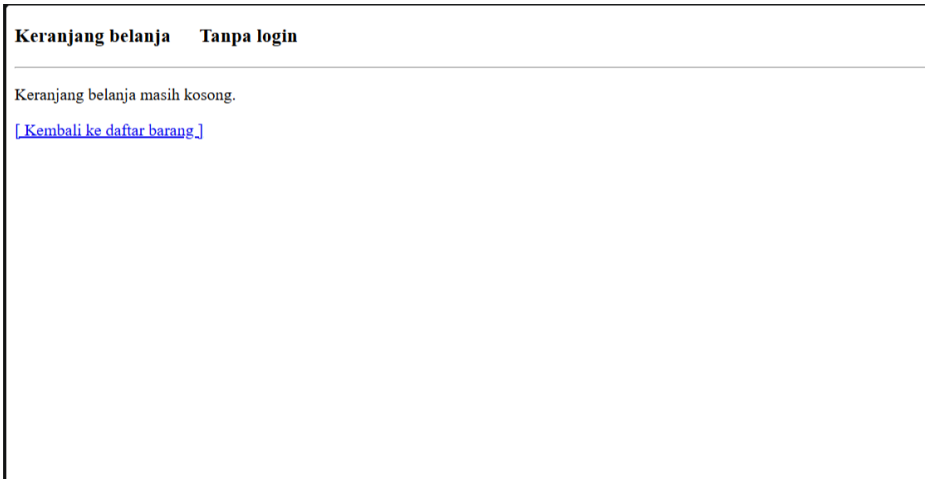
BAGIAN 2: TUGAS 2 - KERANJANG BELANJA TANPA LOGIN

2.1 Hasil Implementasi & Tangkapan Layar (Screenshots)

Tugas 2 mengimplementasikan keranjang belanja tanpa login menggunakan Laravel 13 dan database MySQL db_tugas2 (tabel barang berisi 5 produk). Keranjang hanya menyimpan ID produk dan jumlah di session, sedangkan nama dan harga selalu diambil langsung dari database.



Gambar 5. Halaman daftar barang awal saat keranjang masih kosong [Keranjang (0)]



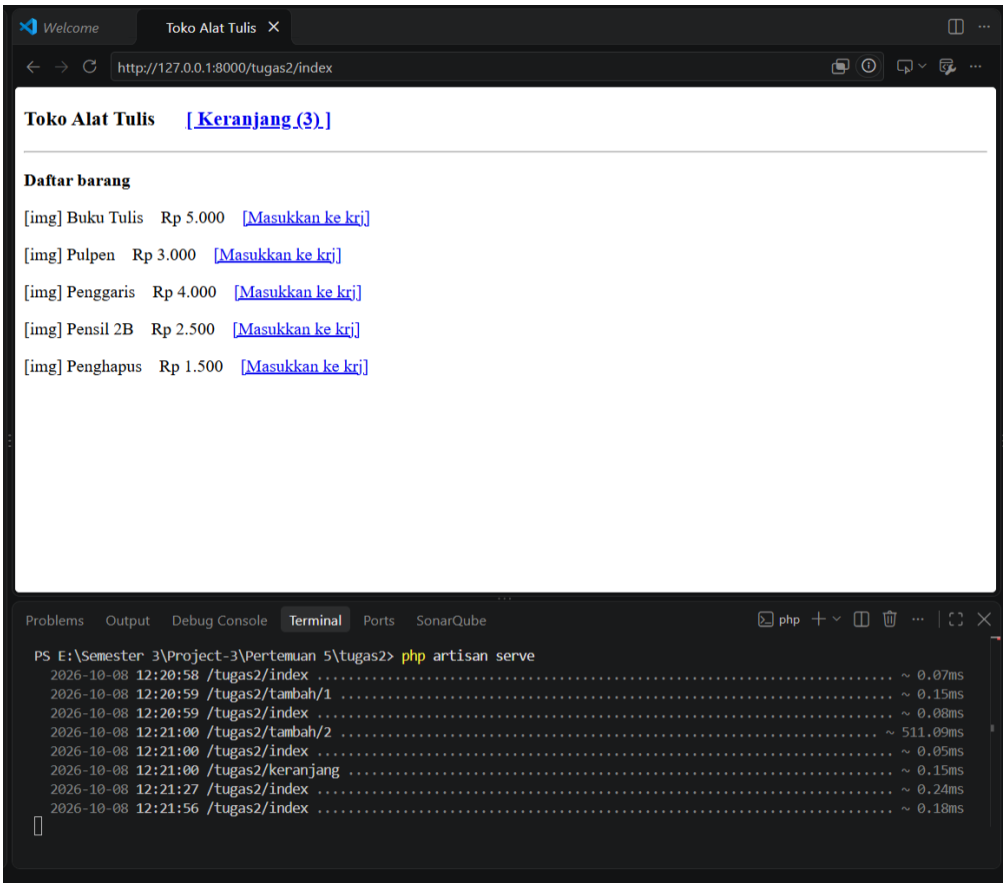
Gambar 6. Halaman keranjang belanja saat belum ada barang yang dimasukkan



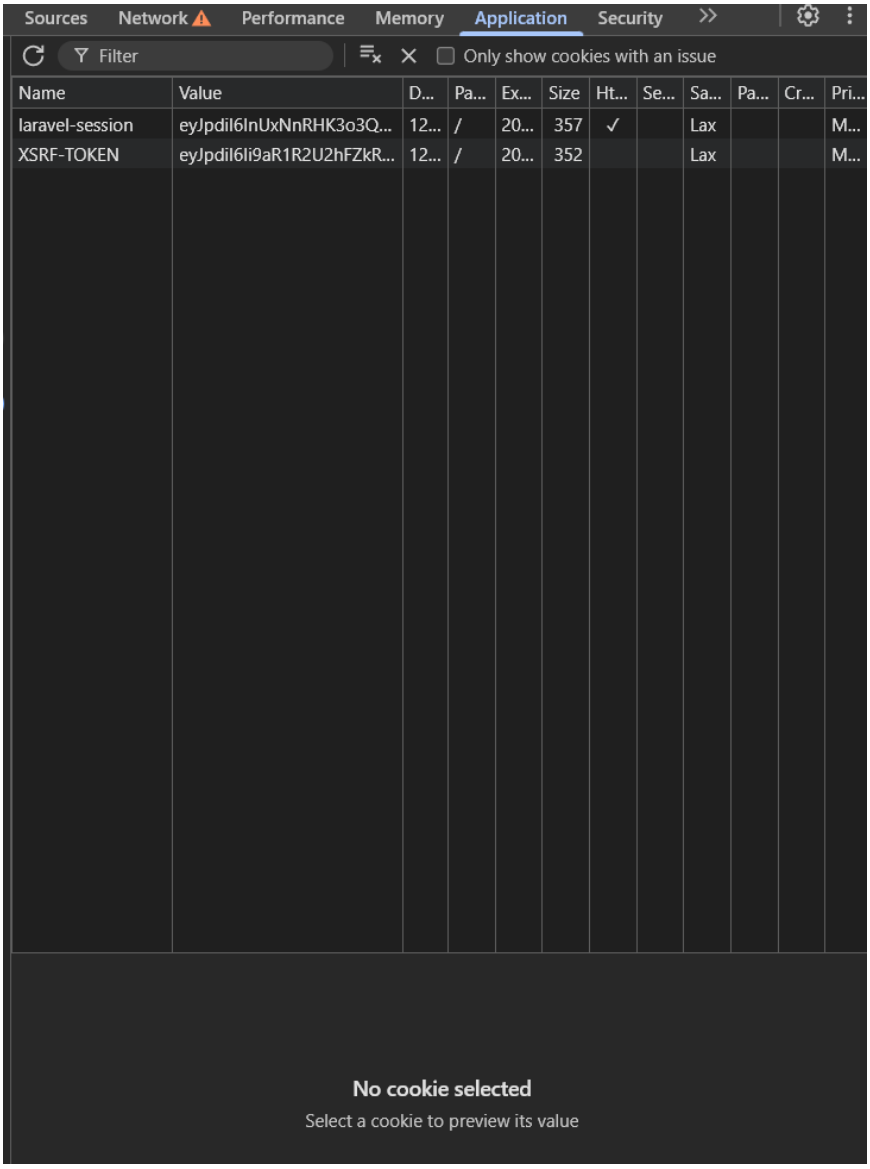
Gambar 7. Halaman daftar barang setelah 3 item dimasukkan [Keranjang (3)]



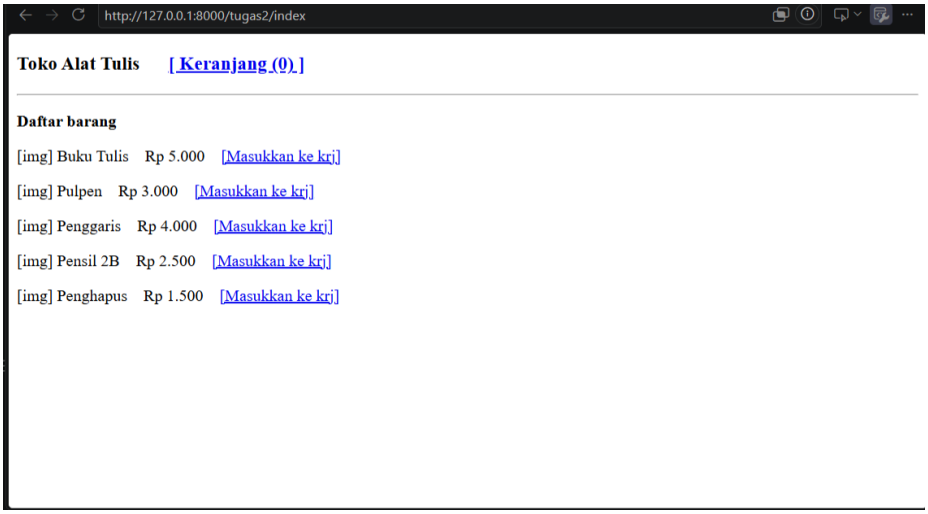
Gambar 8. Halaman keranjang belanja terisi dengan subtotal dan total harga Rp 13.000



Gambar 9. Log terminal php artisan serve saat menangani request penambahan barang



Gambar 10. Lokasi data sesi di DevTools (Application > Cookies > laravel-session)



Gambar 11. Hasil pengujian pertanyaan 3: manipulasi session ID menyebabkan keranjang tereset menjadi 0

2.2 Jawaban Pertanyaan Tugas 2

1. Mekanisme apa yang Anda pilih (session, cookies, atau Local Storage), dan mengapa?

Mekanisme yang saya pilih adalah Session. Alasannya adalah keamanan dan integritas data. Dengan session, browser hanya menyimpan ID sesi acak sedangkan isi data keranjang (ID barang dan jumlah) disimpan di server. Hal ini mencegah pengguna memanipulasi kuantitas atau susunan barang secara sepihak lewat browser.

2. Sebutkan satu kelebihan dan satu kekurangan pilihan Anda dibanding dua mekanisme lainnya.

Kelebihan: Lebih aman karena data keranjang tidak dapat dibaca maupun diedit langsung oleh pengguna melalui inspect element/DevTools browser.
Kekurangan: Memerlukan ruang penyimpanan di sisi server (file atau database) dan data sesi otomatis terhapus saat sesi kedaluwarsa atau browser ditutup.

3. Ubah data keranjang secara manual lewat DevTools (misalnya jumlah menjadi 999) bila memungkinkan pada mekanisme Anda. Apa yang terjadi? Jika tidak bisa diubah, jelaskan alasannya.

Pada mekanisme Session di Laravel, jumlah tidak bisa diubah langsung menjadi 999 lewat DevTools. Hal ini dikarenakan di DevTools (Application > Cookies) hanya tersimpan cookie laravel-session yang berupa string acak terenkripsi, bukan nilai jumlah barang. Ketika nilai cookie tersebut coba diubah secara acak, server Laravel menolaknya karena token enkripsi tidak valid, sehingga saat halaman di-refresh keranjang belanja justru tereset menjadi kosong (Keranjang (0)). Bukti pengujian terlampir pada Gambar 11.

4. Mengapa harga tidak boleh dipercaya dari data yang dikirim atau disimpan di browser? Apa yang harus dilakukan server saat checkout?

Karena semua data di sisi browser/klien sangat mudah diubah oleh pengguna menggunakan DevTools atau manipulasi request HTTP. Jika harga disimpan di browser, pengguna bisa mengubah harga Rp 5.000 menjadi Rp 1. Saat checkout, server wajib mengambil ulang harga asli barang langsung dari database berdasarkan ID barang dan mengalikannya dengan jumlah beli di sisi server.

5. Apa yang terjadi pada keranjang jika pengguna membuka situs dari browser lain? Jelaskan sesuai pilihan Anda.

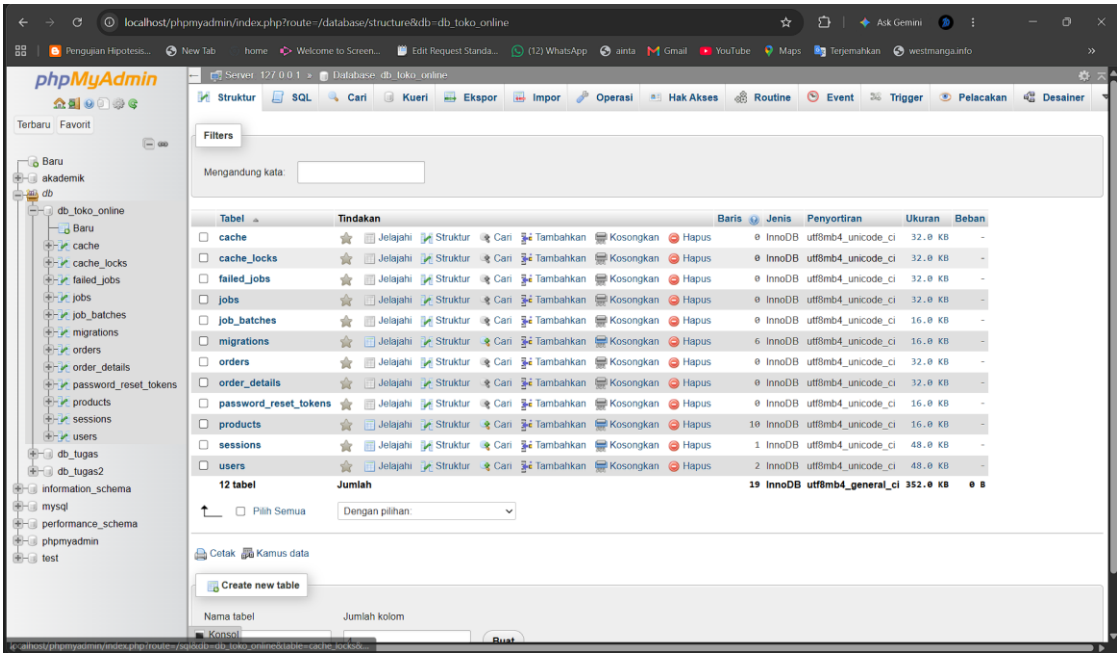
Keranjang belanja akan kosong di browser lain. Karena sistem ini tanpa login, sesi diikat pada cookie browser pertama. Browser yang berbeda memiliki penyimpanan cookie terpisah sehingga mendapatkan session ID baru, dan server menganggapnya sebagai pengunjung yang berbeda.

BAGIAN 3: TUGAS MANDIRI (PR) - PROGRAM TOKO ONLINE

3.1 Deskripsi Tugas & Rancangan Database

Program toko online menggabungkan sistem autentikasi, katalog produk publik, keranjang belanja session terproteksi login, validasi stok, checkout transaksi, dan riwayat pesanan. Sesuai spesifikasi, database db_toko_online memiliki 4 tabel:

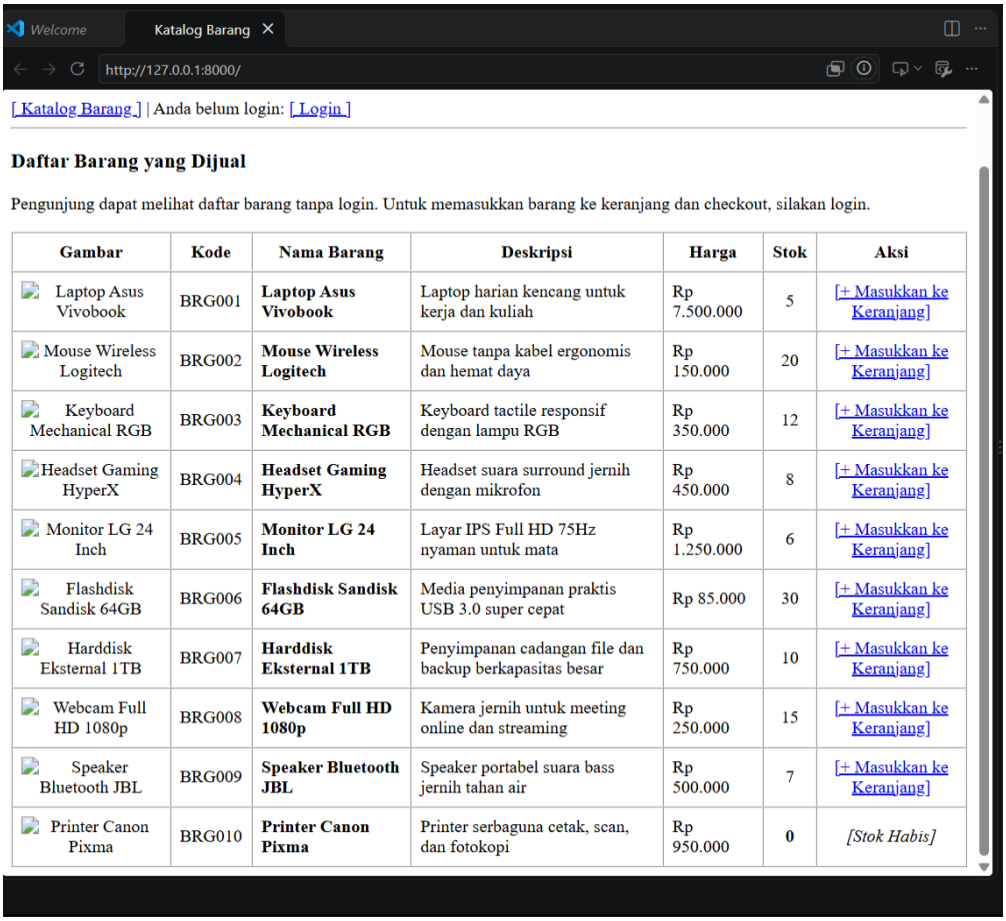
1. Tabel users: id_user (PK, VARCHAR 15), nama_lengkap, email, username, password (hash), no_hp, alamat.
2. Tabel products: id_barang (PK, VARCHAR 10), nama_barang, deskripsi, harga, stok, gambar.
3. Tabel orders: id_order (PK, VARCHAR 15), id_user (FK), tanggal_order, total_harga, alamat_pengiriman.
4. Tabel order_details: Composite PK (id_order, id_barang), harga_satuan (arsip), jumlah_beli.



Gambar 12. Struktur tabel database db_toko_online di phpMyAdmin

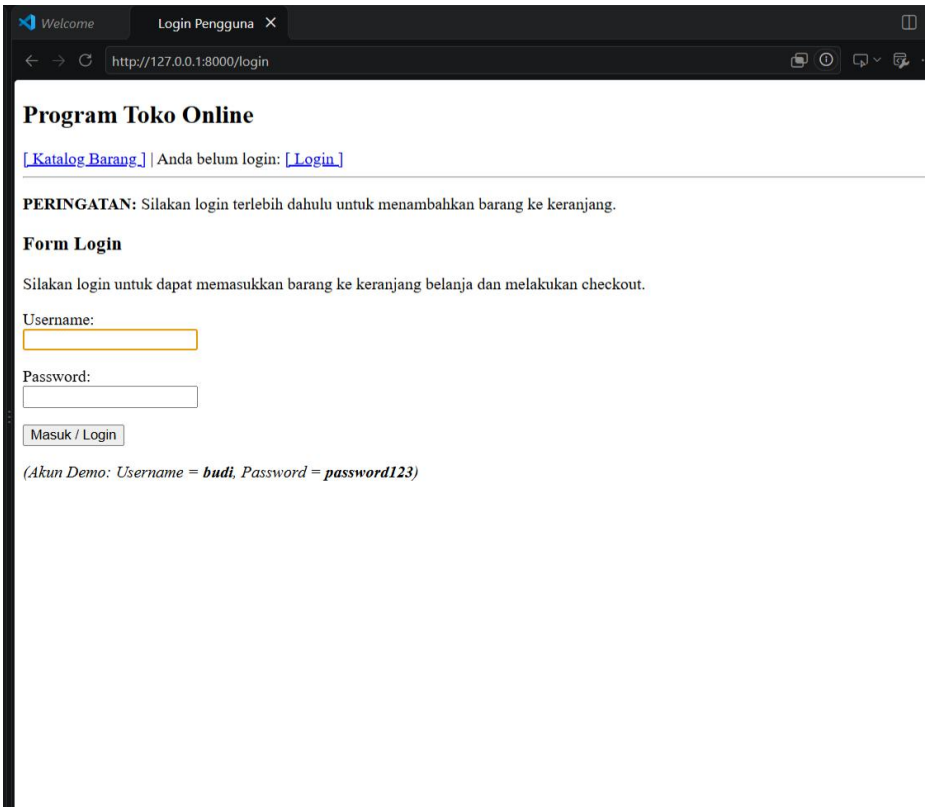
3.2 Ketentuan 1 & 2: Katalog Barang & Pembatasan Pengunjung Belum Login

Pengunjung yang belum login dapat melihat 10 barang di katalog. Barang dengan stok 0 (Printer Canon Pixma) tidak dapat dibeli dan berlabel [Stok Habis].



Gambar 13. Katalog 10 barang saat belum login (stok 0 berstatus [Stok Habis])

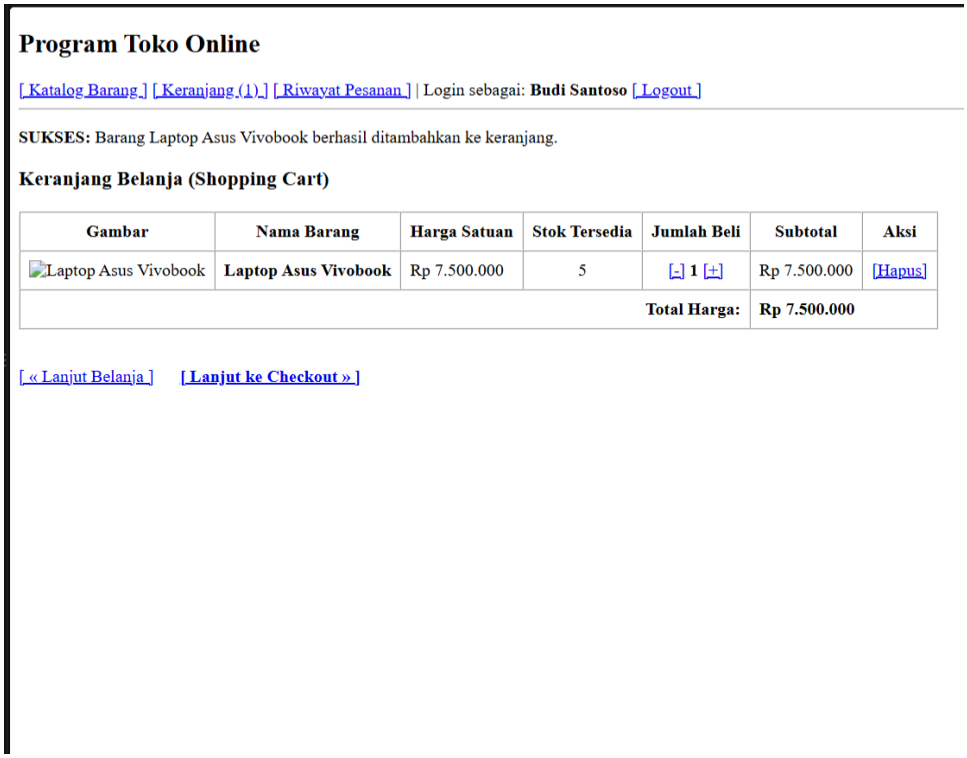
Jika pengunjung belum login mencoba memasukkan barang ke keranjang, sistem otomatis mengalihkan ke form login (/login) disertai pesan peringatan.



Gambar 14. Pengalihan otomatis ke halaman login saat mencoba memasukkan barang tanpa login

3.3 Ketentuan 3: Shopping Cart (Keranjang Belanja) & Validasi Stok

Setelah login (Budi Santoso), pengguna dapat memasukkan barang ke keranjang. Halaman keranjang menampilkan barang, harga satuan, stok tersedia, jumlah beli, subtotal, dan total harga.



Gambar 15. Halaman keranjang belanja setelah barang berhasil dimasukkan

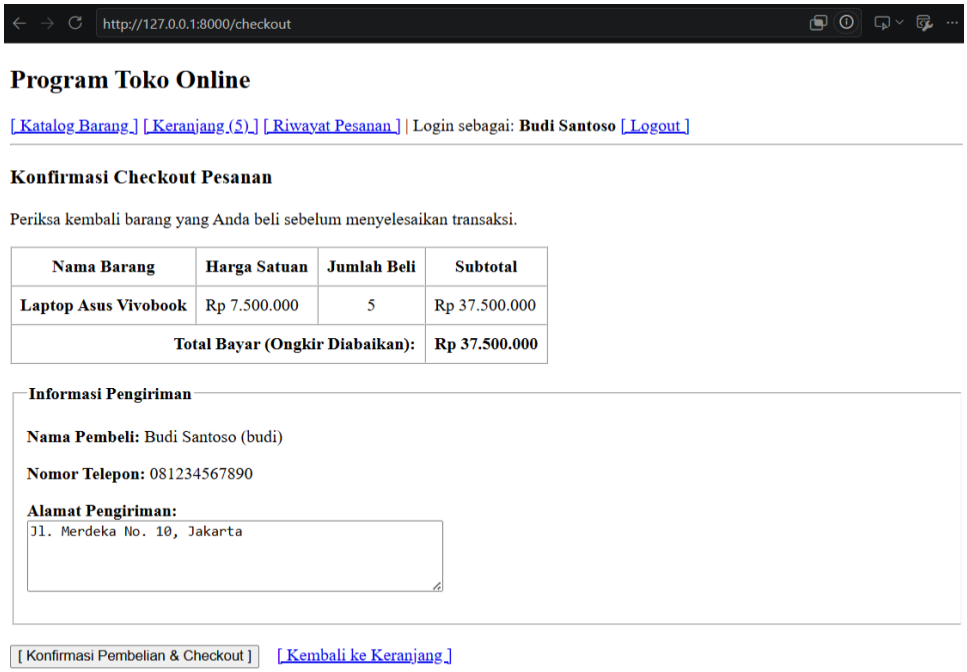
Sistem memvalidasi agar kuantitas beli tidak melebihi stok yang tersedia. Saat tombol [+] ditekan melebihi stok yang ada (stok 5), sistem menampilkan notifikasi error.



Gambar 16. Validasi sistem saat kuantitas keranjang mencoba melebihi stok

3.4 Ketentuan 4: Checkout Pesanan

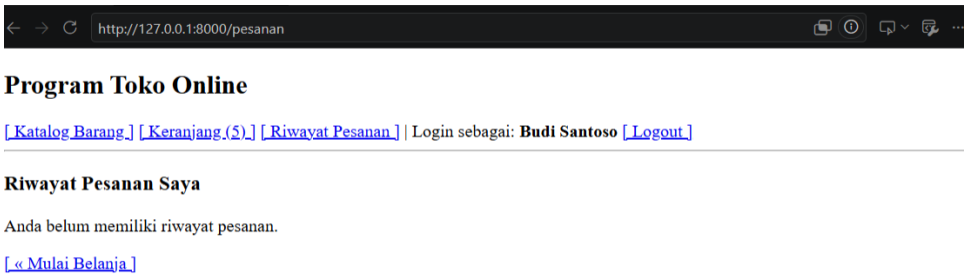
Pembelian hanya sah jika pengguna melakukan checkout dari keranjang. Form checkout menampilkan ringkasan belanja, total bayar (ongkir diabaikan), dan form isian alamat pengiriman.



Gambar 17. Halaman konfirmasi checkout dan isian alamat pengiriman

3.5 Ketentuan 5: Riwayat Pesanan & Pengurangan Stok

Halaman riwayat pesanan sebelum checkout berstatus kosong.



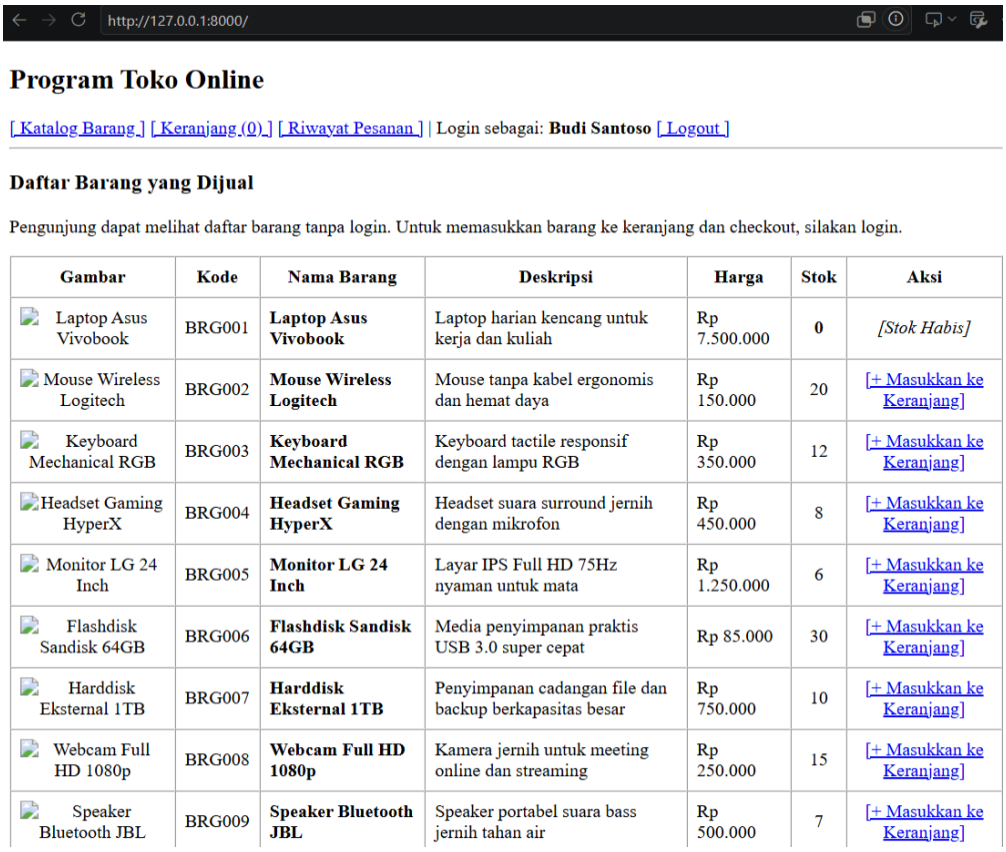
Gambar 18. Halaman riwayat pesanan awal yang belum memiliki transaksi

Setelah checkout dikonfirmasi, sistem membuat data di tabel orders dan order_details, mengurangi stok barang, serta mengosongkan keranjang belanja.



Gambar 19. Riwayat pesanan setelah checkout sukses dan keranjang otomatis kosong (0)

Bukti pengurangan stok dapat dilihat pada katalog: stok Laptop Asus Vivobook yang semula 5 telah berkurang 5 menjadi 0 dan statusnya berubah menjadi [Stok Habis].



Gambar 20. Bukti stok barang di katalog berkurang menjadi 0 dan berstatus [Stok Habis]

3.6 Tabel Verifikasi Skenario Pengujian (Definition of Done)

Tabel verifikasi seluruh skenario pengujian pada program toko online:

ID	Skenario Pengujian	Hasil yang Diharapkan	Status	Keterangan / Bukti
SK-01	Katalog dibuka tanpa login	10 barang tampil lengkap dengan harga dan stok	**PASS**	Gambar 13 (Katalog)
SK-02	Barang stok 0 dibuka di katalog	Tidak ada tombol beli, berlabel [Stok Habis]	**PASS**	Gambar 13 (Printer Pixma)
SK-03	Klik beli barang tanpa login	Ditolak dan dialihkan ke form login	**PASS**	Gambar 14 (Redirect ke /login)
SK-04	Login pengguna berhasil	Header menampilkan nama user dan menu aktif	**PASS**	Gambar 15 (Login Budi)
SK-05	Tambah kuantitas melebihi stok	Sistem memblokir dan memunculkan notifikasi error	**PASS**	Gambar 16 (Validasi stok 5)
SK-06	Checkout pesanan	Order tersimpan ke	**PASS**	Gambar 19

		tabel orders dan order_details		(Pesanan #ORD...)
SK-07	Pengosongan keranjang	Session keranjang otomatis kosong pasca checkout	**PASS**	Gambar 19 (Keranjang (0))
SK-08	Pengurangan stok barang	Stok berkurang di database dan katalog	**PASS**	Gambar 20 (Stok jadi 0)
SK-09	Riwayat pesanan pengguna	Menampilkan riwayat beserta rincian barang	**PASS**	Gambar 19 (Detail pesanan)

3.7 Kesimpulan

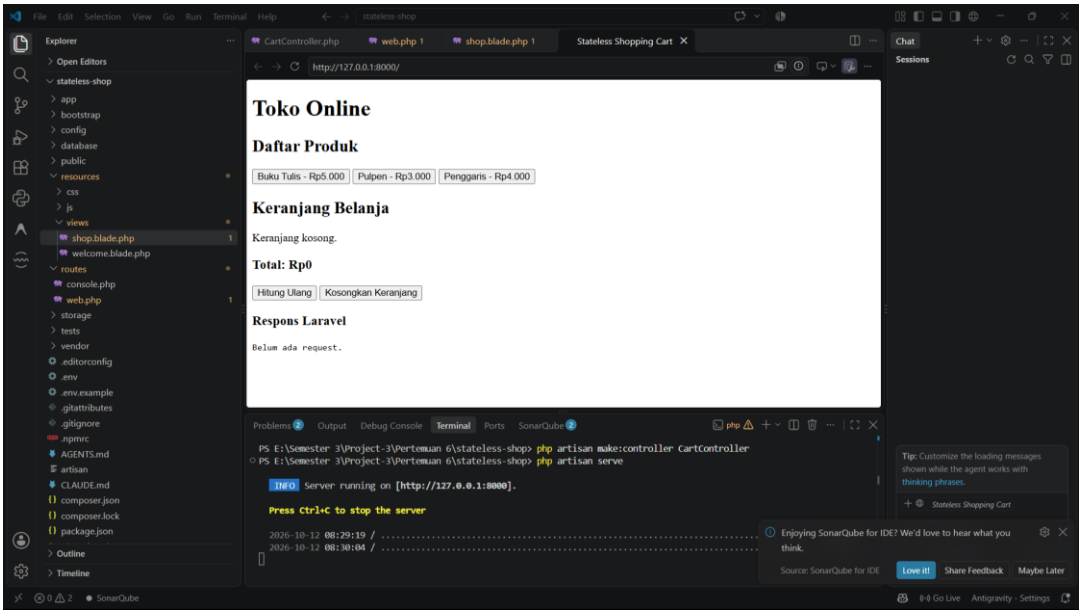
Seluruh tugas pada Modul 4 (Tugas 1 Login Terenkripsi, Tugas 2 Keranjang Belanja Tanpa Login, dan Tugas Mandiri Program Toko Online) telah berhasil diselesaikan dan diuji dengan baik. Penggunaan framework Laravel bersama mekanisme session dan database MySQL memungkinkan implementasi penyimpanan data berbasis web yang aman, terstruktur, dan konsisten.

BAGIAN 4: LINK REPOSITORY

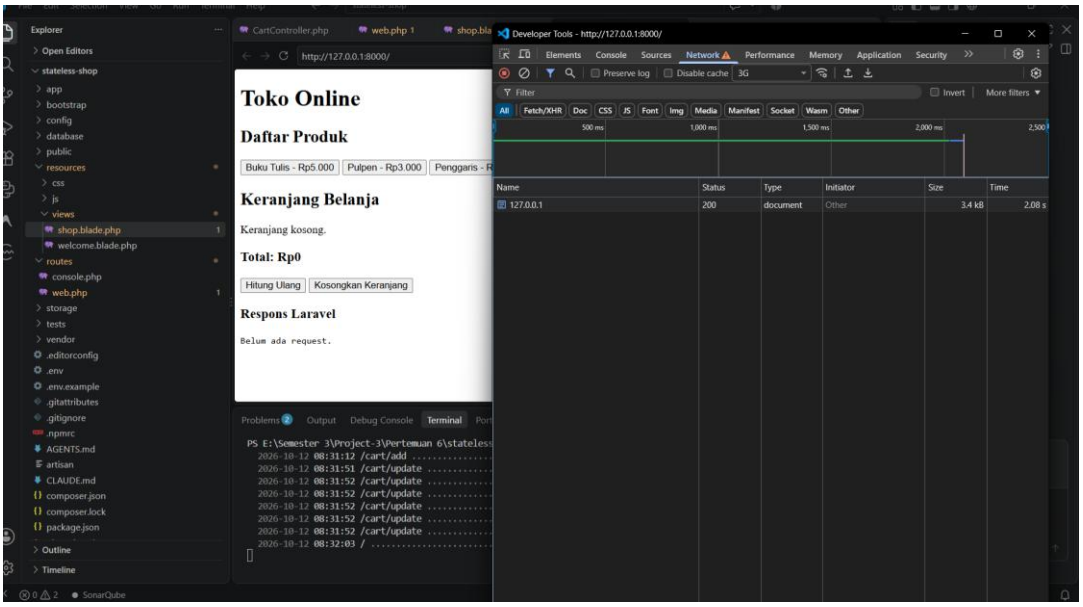
https://github.com/salfffariss/praktikum-modul-04-251511030-Salman

BAGIAN 1: HASIL IMPLEMENTASI & TANGKAPAN LAYAR (SCREENSHOTS)

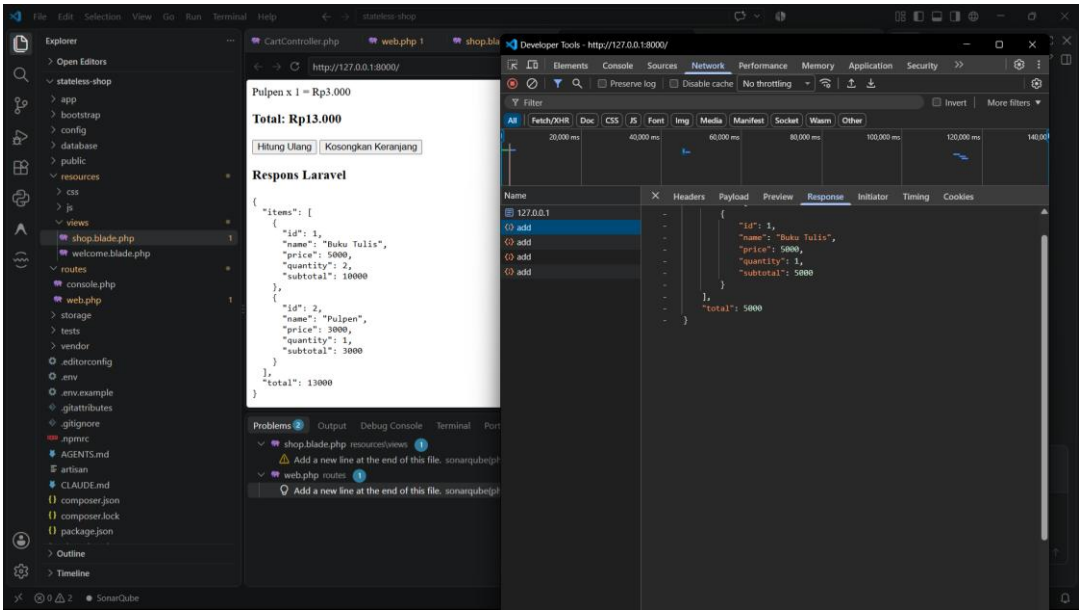
2.1 Implementasi dan Pengujian pada Web Browser



Gambar 1. Inisialisasi proyek stateless-shop pada VS Code dan tampilan awal toko online saat keranjang masih kosong

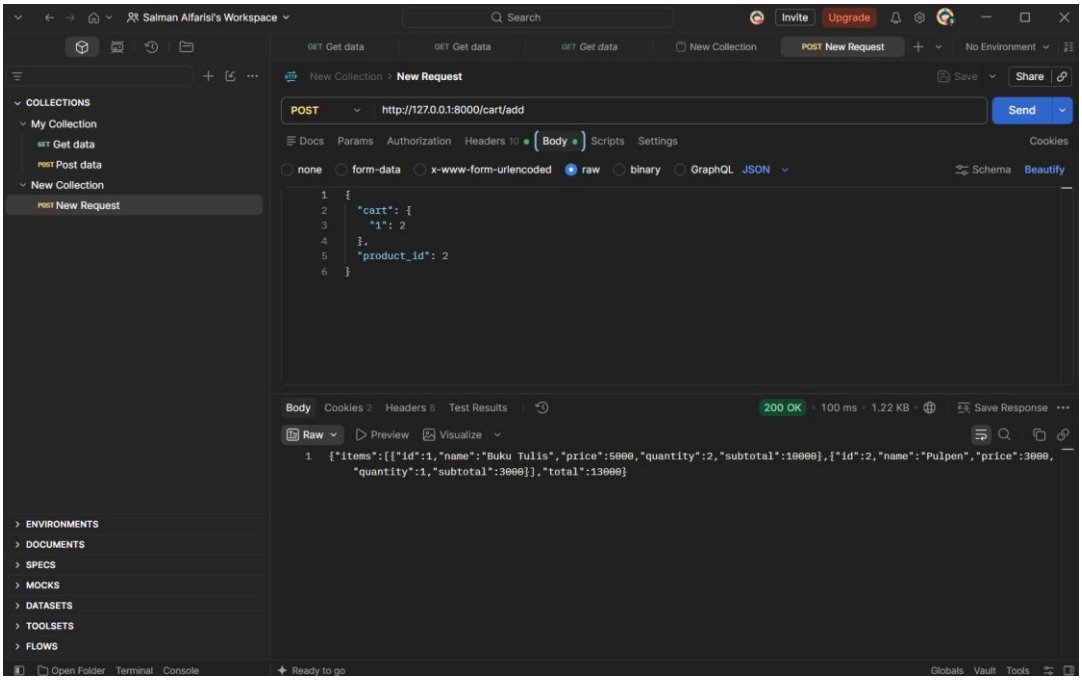


Gambar 2. Kondisi halaman toko online setelah di-refresh (F5) yang membuktikan keranjang belanja tereset menjadi kosong



Gambar 3. Hasil penambahan 2 Buku Tulis dan 1 Pulpen beserta inspeksi respons JSON pada DevTools Network

2.2 Pengujian Operasi Stateless Menggunakan Postman



Gambar 4. Pengujian request POST /cart/add melalui Postman dengan payload JSON

Pada Gambar 4, pengujian Postman mengirimkan payload body: {"cart": {"1": 2}, "product_id": 2}. Server Laravel menerima data state keranjang yang dikirimkan (2 Buku Tulis) lalu menambahkan 1 Pulpen (product_id: 2). Server merespons dengan status 200 OK dan mengembalikan kalkulasi JSON berisi rincian barang serta total Rp 13.000. Pengujian pengiriman ulang dengan payload yang sama persis menghasilkan respon yang selalu identik, membuktikan bahwa server tidak mengakumulasikan riwayat pemanggilan sebelumnya.

BAGIAN 2: DISKUSI DAN EVALUASI

1. Di baris kode mana browser mengirim state keranjang ke Laravel?

Pada file resources/views/shop.blade.php, data keranjang dikirim saat memanggil post, pada fungsi add() dan post pada fungsi recalculate(). Secara fisik melalui jaringan HTTP, state keranjang dialirkan pada baris body: JSON.stringify(payload) menggunakan Fetch API.

2. Mengapa controller tidak perlu membaca session untuk menghitung keranjang?

Karena server bersifat stateless, di mana setiap request dari browser sudah membawa seluruh data keranjang secara lengkap di dalam payload HTTP. Controller cukup membaca data payload tersebut dan menghitung total berdasarkan daftar produk lokal tanpa bergantung pada session server.

3. Apa yang terjadi jika request dikirim ulang dengan payload yang sama?

Respons server akan selalu sama persis (bersifat idempotent). Karena server stateless tidak menyimpan riwayat transaksi dan tidak mengakumulasikan pemanggilan sebelumnya, melainkan hanya memproses snapshot data payload yang dikirim saat itu.

4. Mengapa keranjang kosong setelah halaman di-refresh?

Karena variabel keranjang (let cart = {}) hanya tersimpan di memori kerja (RAM) JavaScript browser tanpa disimpan ke localStorage, cookie, maupun session server. Saat halaman di-refresh (F5), memori JavaScript dibersihkan sehingga variabel cart otomatis tereset kembali ke {}.

5. Apakah penggunaan middleware session Laravel berarti keranjang ini disimpan dalam session? Jelaskan.

Tidak. Middleware session aktif secara default untuk kebutuhan internal framework (seperti verifikasi token CSRF). Pada CartController.php, tidak ada pemanggilan session()->put() atau session()->get(); data keranjang murni diproses langsung dari request payload dan dikembalikan via JSON.

6. Apa risiko keamanan jika server mempercayai harga yang dikirim browser?

Terjadi risiko manipulasi harga (price tampering). Pengguna dapat dengan mudah mengubah nilai harga menjadi sangat murah lewat DevTools atau Postman sebelum dikirim. Oleh karena itu, harga satuan dan perhitungan total wajib selalu ditentukan langsung oleh server berdasarkan data katalog resmi.

BAGIAN 3: LESSON-LEARNED (STATEFUL VS STATELESS)

Berdasarkan perbandingan implementasi antara Modul Praktikum sebelumnya (Stateful menggunakan Laravel Session) dan praktikum ini (Stateless Shopping Cart), diperoleh pemahaman mendalam mengenai arsitektur sistem web:

1. Lokasi Penyimpanan State:

Pada sistem stateful, server bertanggung jawab menyimpan status transaksi pengguna (session) dan mengikatnya dengan cookie session ID di browser. Pada sistem stateless, server sama sekali tidak menyimpan riwayat data pengguna; browser wajib membawa seluruh konteks atau state transaksi di setiap request HTTP.

2. Beban Komputasi dan Memori (Server Overhead):

Sistem stateful mengonsumsi memori dan ruang penyimpanan server seiring bertambahnya jumlah pengguna bersamaan (concurrent users). Sebaliknya, sistem stateless membebaskan server dari beban memori sesi sehingga server bekerja sangat ringan dan efisien, meskipun konsekuensinya ukuran payload HTTP yang dikirimkan klien menjadi sedikit lebih besar.